Symbiosis Institute of Technology, Pune

Faculty of Engineering
CSE- Academic Year 2026-27
Java Programming – Lab Batch 2024-28

Lab Assignment No:- 24	
Name of Student	Abishek Yadav
PRN No.	25070122014
Batch	A1
Class	CSE
Academic Year & Semester	2026-2027/3RD
Date of Submission	07-10-26
Title of Assignment:	Develop a Java GUI application integrated with JDBC to perform database operations.
Theory:	Java Swing provides GUI components such as JFrame, JLabel, JTextField, and JButton for building desktop applications. When a Swing application is integrated with JDBC, the values entered by the user in the form are passed to SQL statements, so that database operations such as INSERT, UPDATE, DELETE, and retrieval of records (SELECT) can be performed from buttons using event handling. PreparedStatement is used to execute parameterized queries, and JOptionPane is used to display result and error messages to the user.
Practice Questions	1. Create a Library Management System GUI connected with database.

CODE:

```
import javax.swing.*;
import java.awt.*;
import java.sql.*;

public class LibraryManagementGUI extends JFrame
{
    JTextField bookIdField, studentNameField,
               issueDateField, returnDateField;
    JButton addButton, viewButton, updateButton, deleteButton;
    Connection con;

    public LibraryManagementGUI()
    {
        setTitle("Library Management System");
        setSize(600, 450);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new GridLayout(6, 2, 10, 10));

        add(new JLabel("Book ID:"));
        bookIdField = new JTextField();
        add(bookIdField);

        add(new JLabel("Student Name:"));
        studentNameField = new JTextField();
        add(studentNameField);

        add(new JLabel("Issue Date (YYYY-MM-DD):"));
        issueDateField = new JTextField();
        add(issueDateField);

        add(new JLabel("Return Date (YYYY-MM-DD):"));
        returnDateField = new JTextField();
        add(returnDateField);

        addButton = new JButton("Add Record");
        viewButton = new JButton("View Records");
        updateButton = new JButton("Update Record");
        deleteButton = new JButton("Delete Record");

        add(addButton);
        add(viewButton);
```

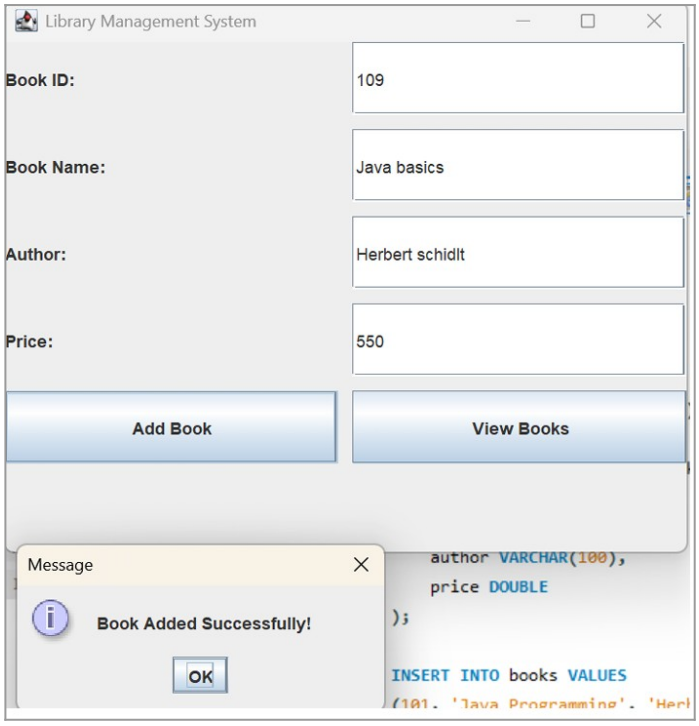
```
        add(updateButton);
        add(deleteButton);

        connectDatabase();
        setVisible(true);
    }

    void connectDatabase()
    {
        try
        {
            con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/book_issue",
                "root", "abishek@123");
            JOptionPane.showMessageDialog(this,
                "Database Connected Successfully!");
        }
        catch (SQLException e)
        {
            JOptionPane.showMessageDialog(this,
                "Database Connection Failed!\n" + e.getMessage());
        }
    }

    public static void main(String[] args)
    {
        new LibraryManagementGUI();
    }
}
```

OUTPUT:



QUESTION	2. Create a Book Issue Tracking System GUI application using Java Swing connected to a database. The system should store book ID, student name, issue date, and return date, and allow record management.

CODE:

```
import javax.swing.*;
import java.awt.*;
import java.sql.*;

public class BookIssueTrackingGUI extends JFrame
{
    JTextField id, name, issue, ret;
    Connection con;

    BookIssueTrackingGUI()
    {
        setTitle("Book Issue Tracking System");
        setSize(500, 300);
```

```
        setLayout(new GridLayout(6, 2, 10, 10));
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);

        add(new JLabel("Book ID"));
        id = new JTextField();
        add(id);

        add(new JLabel("Student Name"));
        name = new JTextField();
        add(name);

        add(new JLabel("Issue Date (YYYY-MM-DD)"));
        issue = new JTextField();
        add(issue);

        add(new JLabel("Return Date (YYYY-MM-DD)"));
        ret = new JTextField();
        add(ret);

        JButton addBtn = new JButton("Add");
        JButton viewBtn = new JButton("View");
        JButton updateBtn = new JButton("Update");
        JButton deleteBtn = new JButton("Delete");

        add(addBtn);
        add(viewBtn);
        add(updateBtn);
        add(deleteBtn);

        connect();

        addBtn.addActionListener(e -> addRecord());
        viewBtn.addActionListener(e -> viewRecord());
        updateBtn.addActionListener(e -> updateRecord());
        deleteBtn.addActionListener(e -> deleteRecord());

        setVisible(true);
    }

    void connect()
    {
        try
        {
            con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/book_issue",
                "root", "abishek@123");
            System.out.println("Database Connected!");
        }
        catch (Exception e)
        {
            JOptionPane.showMessageDialog(this,
                "Connection Error: " + e.getMessage());
        }
    }

    void addRecord()
    {
        try
        {
            PreparedStatement p = con.prepareStatement(
                "INSERT INTO book_issue VALUES (?, ?, ?, ?)");
            p.setInt(1, Integer.parseInt(id.getText()));
            p.setString(2, name.getText());
            p.setDate(3, Date.valueOf(issue.getText()));
            p.setDate(4, Date.valueOf(ret.getText()));
            p.executeUpdate();
            JOptionPane.showMessageDialog(this, "Record Added!");
        }
        catch (Exception e)
        {
            JOptionPane.showMessageDialog(this,
                "Error: " + e.getMessage());
        }
    }

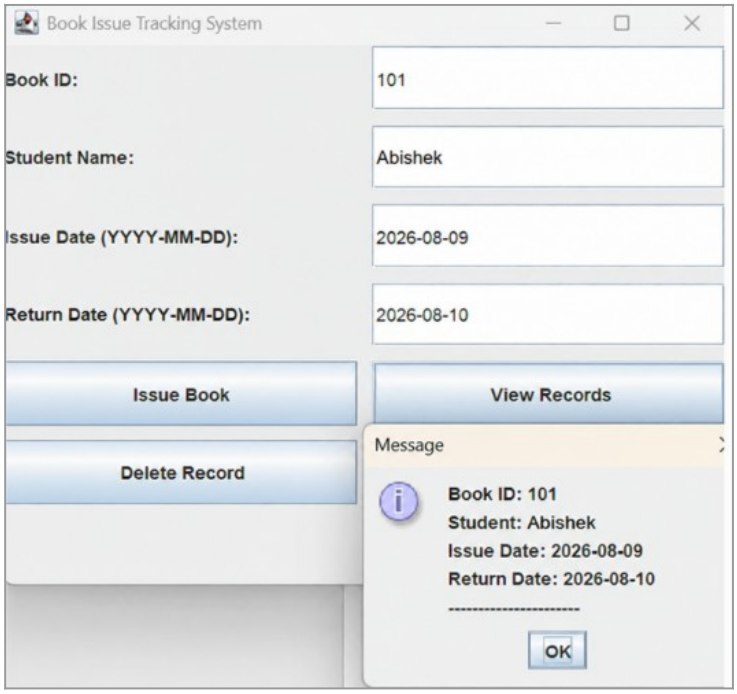
    void viewRecord()
    {
        try
        {
            ResultSet r = con.createStatement()
                .executeQuery("SELECT * FROM book_issue");
            String s = "ID Name Issue Date Return Date\n";
```

```
            while (r.next())
            {
                s += r.getInt(1) + " "
                   + r.getString(2) + " "
                   + r.getDate(3) + " "
                   + r.getDate(4) + "\n";
            }
            JOptionPane.showMessageDialog(this, s);
        }
        catch (Exception e)
        {
            JOptionPane.showMessageDialog(this,
                "Error: " + e.getMessage());
        }
    }

    void updateRecord()
    {
        try
        {
            PreparedStatement p = con.prepareStatement(
                "UPDATE book_issue SET student_name=?, " +
                "issue_date=?, return_date=? WHERE book_id=?");
            p.setString(1, name.getText());
            p.setDate(2, Date.valueOf(issue.getText()));
            p.setDate(3, Date.valueOf(ret.getText()));
            p.setInt(4, Integer.parseInt(id.getText()));
            int n = p.executeUpdate();
            JOptionPane.showMessageDialog(this,
                n > 0 ? "Record Updated!" : "Book ID Not Found!");
        }
        catch (Exception e)
        {
            JOptionPane.showMessageDialog(this,
                "Error: " + e.getMessage());
        }
    }

    void deleteRecord()
    {
        try
        {
            PreparedStatement p = con.prepareStatement(
                "DELETE FROM book_issue WHERE book_id=?");
            p.setInt(1, Integer.parseInt(id.getText()));
            int n = p.executeUpdate();
            JOptionPane.showMessageDialog(this,
                n > 0 ? "Record Deleted!" : "Book ID Not Found!");
        }
        catch (Exception e)
        {
            JOptionPane.showMessageDialog(this,
                "Error: " + e.getMessage());
        }
    }

    public static void main(String[] args)
    {
        new BookIssueTrackingGUI();
    }
}
```

OUTPUT:	

Conclusion	The Java Swing applications were successfully integrated with JDBC. The Library Management System GUI connected to the database, and the Book Issue Tracking System was able to add, view, update, and delete book issue records using PreparedStatement. This helped in understanding how a GUI front end can be connected with a database.